

QuickBite: SQL Query Practice (Basic → Advanced)

This guide has 34 exercises on the **QuickBite** database. They start with simple `SELECT`s and build up to window functions and transactions. Every solution was run against the real database, and the output shown is the actual result.

How to practise

1. Build the database once:

```
./run_all.sh
```

2. Open an SQL prompt:

```
psql -d quickbite
```

3. Read the question and try writing the query **yourself** first. Use the hint only if you're stuck.
4. Open **Solution** and compare your query and output. If your rows match, your query is correct, even if it's written differently.

The sample data is fixed, so you should get exactly the same rows on every run. If you break the data while experimenting, rerun `./run_all.sh` to reset it.

Cheat sheet: the tables

Table	Columns
restaurant	restaurant_id, name, cuisine, location, address, phone, rating, is_active
menu_item	menu_item_id, restaurant_id → restaurant, name, category, price, is_veg, is_available
customer	customer_id, full_name, email, phone, address, location, joined_on
delivery_agent	delivery_agent_id, full_name, phone, vehicle_type, vehicle_no, home_zone, is_active
food_order	order_id, customer_id → customer, restaurant_id → restaurant, delivery_agent_id → delivery_agent, order_time, delivery_address, status, delivered_time
order_item	order_id → food_order, menu_item_id → menu_item, quantity, unit_price (<i>one row per dish in an order</i>)
payment	payment_id, order_id → food_order, method, status, transaction_ref, paid_at
v_order_total (view)	order_id, restaurant_id, customer_id, status, order_time, order_total

The order statuses are: Placed, Preparing, Out for Delivery, Delivered and Cancelled.

Contents

- [Level 1 — Basics](#): exercises 1–8
- [Level 2 — Aggregates & GROUP BY](#): exercises 9–14
- [Level 3 — JOINS](#): exercises 15–20
- [Level 4 — Subqueries](#): exercises 21–25
- [Level 5 — Advanced \(CTEs, window functions, CASE, dates, transactions\)](#): exercises 26–34

- [Self-check challenges \(no solutions\)](#)
-

Level 1 — Basics

Filtering and sorting rows from a single table: `SELECT`, `WHERE`, `ORDER BY`, `DISTINCT`, `LIKE`, `IN`, `IS NULL`, `LIMIT`.

1. List every restaurant

Show the name, cuisine and location of every restaurant, alphabetically by name.

Hint

```
SELECT ... FROM ... ORDER BY
```

Solution

```
SELECT name, cuisine, location
FROM   restaurant
ORDER BY name;
```

Output:

name	cuisine	location
Amritsari Kulcha House	North Indian	Kharghar
Biryani Darbar	Mughlai	Andheri West
Dragon Wok	Chinese	Powai
La Pizzeria Napoli	Italian	Bandra West
Madras Filter Cafe	South Indian	Thane West
Malvani Tadka	Malvani	Vashi
Mumbai Chaat Corner	Street Food	Vashi
Punjab Da Dhaba	North Indian	Kharghar
The Green Bowl	Continental	Powai

Udupi Sagar		South Indian		Vashi
Wok Express		Chinese		Vashi

(11 rows)

2. Cheap veg dishes

List veg dishes that cost less than ₹100. Show the dish name and price, cheapest first.

Hint

Two conditions in `WHERE`, joined by `AND`. Booleans can be tested directly.

Solution

```
SELECT name, price
FROM   menu_item
WHERE  is_veg AND price < 100
ORDER BY price, name;
```

Output:

name	price
Vada Pav	30.00
Masala Chaas	40.00
Filter Coffee	50.00
Butter Naan	60.00
Madras Filter Coffee	60.00
Pani Puri	60.00
Sev Puri	70.00
Idli Sambar	80.00
Sol Kadhi	80.00
Medu Vada	90.00
Sweet Lassi	90.00

(11 rows)

3. Distinct cuisines

Which cuisines does QuickBite offer? Show each cuisine once.

Hint

`DISTINCT`

Solution

```
SELECT DISTINCT cuisine
FROM   restaurant
ORDER BY cuisine;
```

Output:

```
    cuisine
-----
Chinese
Continental
Italian
Malvani
Mughlai
North Indian
South Indian
Street Food
(8 rows)
```

4. Pattern matching

Find customers whose full name starts with 'A'.

Hint

`LIKE 'A%'`. `%` matches any number of characters.

Solution

```
SELECT customer_id, full_name, location
FROM   customer
WHERE  full_name LIKE 'A%'
ORDER  BY full_name;
```

Output:

customer_id	full_name	location
1	Aarav Sharma	Kharghar
13	Aditya Rao	Andheri West
6	Ananya Iyer	Bandra West
9	Arjun Reddy	Thane West

(4 rows)

5. IN and comparison

Show restaurants in Vashi **or** Powai with a rating of at least 4.2.

Hint

```
location IN ('Vashi','Powai')
```

Solution

```
SELECT name, location, rating
FROM   restaurant
WHERE  location IN ('Vashi', 'Powai')
      AND rating >= 4.2
ORDER  BY rating DESC;
```

Output:

name	location	rating
The Green Bowl	Powai	4.4
Udupi Sagar	Vashi	4.2

(2 rows)

6. Working with NULL

Which delivery agents have no vehicle number?

Hint

`= NULL` never matches anything. Use `IS NULL`.

Solution

```
SELECT full_name, vehicle_type
FROM   delivery_agent
WHERE  vehicle_no IS NULL;
```

Output:

full_name	vehicle_type
Anil Kamble	Bicycle

(1 row)

7. Top-N with LIMIT

Show the 3 most expensive dishes on the platform.

Hint

`ORDER BY ... DESC LIMIT 3`

Solution

```
SELECT name, price
FROM   menu_item
ORDER  BY price DESC
LIMIT 3;
```

Output:

name	price
Pepperoni Pizza	499.00
Grilled Chicken Steak	450.00
Mutton Biryani	420.00
(3 rows)	

8. Date range

List orders placed from 10 Sep 2026 to 12 Sep 2026, inclusive.

Hint

Use `>=` the first day and `<` the day *after* the last day, because timestamps carry times.

Solution

```
SELECT order_id, order_time, status
FROM   food_order
WHERE  order_time >= '2026-09-10'
      AND order_time <  '2026-09-13'
ORDER  BY order_time;
```

Output:

order_id	order_time	status
34	2026-09-10 20:25:00	Delivered
4	2026-09-10 21:15:00	Delivered
38	2026-09-11 09:05:00	Delivered
24	2026-09-11 21:00:00	Delivered
14	2026-09-12 20:00:00	Delivered
(5 rows)		

Level 2 — Aggregates & GROUP BY

Turning many rows into one number per group: COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING.

9. COUNT per group

How many menu items does each restaurant have? Show the restaurant id and the count.

Hint

```
GROUP BY restaurant_id
```

Solution

```
SELECT restaurant_id, COUNT(*) AS items
FROM menu_item
GROUP BY restaurant_id
ORDER BY restaurant_id;
```

Output:

restaurant_id	items
1	5
2	5
3	5
4	5
5	5
6	5
7	5
8	5
9	5
10	5
11	3

(11 rows)

10. AVG with ROUND

Show the average menu price per category, rounded to 2 decimals, highest first.

Hint

```
ROUND (AVG (price), 2)
```

Solution

```
SELECT category, ROUND (AVG (price), 2) AS avg_price, COUNT (*)
FROM menu_item
GROUP BY category
ORDER BY avg_price DESC;
```

Output:

category	avg_price	items
Main Course	275.71	17
Rice	263.33	6
Starter	247.27	11
Dessert	179.50	2
Bread	110.00	3
Snack	88.43	7
Beverage	87.14	7

(7 rows)

11. Count by status

How many orders are in each status?

Hint

Group by `status`.

Solution

```
SELECT status, COUNT(*) AS orders
FROM   food_order
GROUP  BY status
ORDER  BY orders DESC;
```

Output:

status	orders
Delivered	45
Cancelled	2
Preparing	1
Out for Delivery	1

(4 rows)

12. HAVING filters groups

Which restaurants received **more than 5** orders, in any status?

Hint

`WHERE` filters rows *before* grouping. `HAVING` filters groups *after*.

Solution

```
SELECT restaurant_id, COUNT(*) AS orders
FROM   food_order
GROUP  BY restaurant_id
HAVING COUNT(*) > 5
ORDER  BY orders DESC;
```

Output:

restaurant_id	orders
1	7
5	6
6	6

(3 rows)

13. MIN and MAX together

For each restaurant, show its cheapest and most expensive dish price, and the gap between them.

Hint

Several aggregates can go in one `SELECT`.

Solution

```
SELECT restaurant_id,
       MIN(price)           AS cheapest,
       MAX(price)           AS priciest,
       MAX(price) - MIN(price) AS price_range
FROM   menu_item
GROUP  BY restaurant_id
ORDER  BY price_range DESC;
```

Output:

restaurant_id	cheapest	priciest	price_range
11	80.00	420.00	340.00
4	179.00	499.00	320.00
5	110.00	420.00	310.00
1	60.00	340.00	280.00
10	180.00	450.00	270.00
7	110.00	280.00	170.00
6	30.00	140.00	110.00
9	150.00	250.00	100.00

8		60.00		160.00		100.00
2		50.00		130.00		80.00
3		180.00		260.00		80.00

(11 rows)

14. SUM over a table

How many dish units have been sold in total across all order lines, and how many order lines are there?

Hint

`SUM(quantity)` and `COUNT(*)`

Solution

```
SELECT SUM(quantity) AS total_units, COUNT(*) AS order_lines
FROM   order_item;
```

Output:

total_units		order_lines
159		110

(1 row)

Level 3 — JOINS

Combining tables through their keys: `INNER JOIN`, `LEFT JOIN`, `self-joins`, and joining to a view.

15. INNER JOIN, 3 tables

Show the first 8 orders with the customer's name and the restaurant's name.

Hint

Join `food_order` to `customer` and to `restaurant` on their keys.

Solution

```
SELECT fo.order_id, c.full_name AS customer, r.name AS restaurant
FROM food_order fo
JOIN customer c ON c.customer_id = fo.customer_id
JOIN restaurant r ON r.restaurant_id = fo.restaurant_id
ORDER BY fo.order_id
LIMIT 8;
```

Output:

order_id	customer	restaurant	status
1	Aarav Sharma	Punjab Da Dhaba	Delivered
2	Rohan Mehta	Punjab Da Dhaba	Delivered
3	Karan Malhotra	Punjab Da Dhaba	Delivered
4	Neha Gupta	Punjab Da Dhaba	Delivered
5	Meera Pillai	Punjab Da Dhaba	Delivered
6	Aarav Sharma	Punjab Da Dhaba	Delivered
7	Priya Nair	Udupi Sagar	Delivered
8	Sneha Kulkarni	Udupi Sagar	Delivered
(8 rows)			

16. Order lines with totals

For order #1, list each dish, its quantity, unit price and line total.

Hint

```
quantity * unit_price
```

Solution

```
SELECT m.name AS dish, oi.quantity, oi.unit_price,
       oi.quantity * oi.unit_price AS line_total
FROM   order_item oi
JOIN   menu_item m ON m.menu_item_id = oi.menu_item_id
WHERE  oi.order_id = 1
ORDER  BY line_total DESC;
```

Output:

dish	quantity	unit_price	line_total
Butter Chicken	1	340.00	340.00
Butter Naan	4	60.00	240.00
Sweet Lassi	2	90.00	180.00
(3 rows)			

17. LEFT JOIN to find missing rows

Which restaurants have **never** received an order?

Hint

`LEFT JOIN` keeps every restaurant. Restaurants with no orders get `NULLs` on the order side.

Solution

```
SELECT r.name
FROM   restaurant r
LEFT   JOIN food_order fo ON fo.restaurant_id = r.restaurant_id
WHERE  fo.order_id IS NULL;
```

Output:

name

```
-----  
Malvani Tadka  
(1 row)
```

18. JOIN + GROUP BY

For every delivery agent, how many *different* restaurants have they delivered for?

Hint

```
COUNT(DISTINCT ...)
```

Solution

```
SELECT da.full_name, COUNT(DISTINCT fo.restaurant_id) AS  
FROM   delivery_agent da  
JOIN   food_order fo ON fo.delivery_agent_id = da.delivery_agent_id  
GROUP BY da.delivery_agent_id, da.full_name  
ORDER BY restaurants_served DESC, da.full_name;
```

Output:

full_name	restaurants_served
Anil Kamble	3
Suresh Patil	3
Imran Shaikh	2
Manoj Tiwari	2
Ramesh Yadav	2
Deepak Chauhan	1
Ganesh More	1
Joseph D'Souza	1

(8 rows)

19. SELF JOIN

List pairs of restaurants that serve the same cuisine.

Hint

Join restaurant to itself. Use `a.restaurant_id < b.restaurant_id` so each pair appears once.

Solution

```
SELECT a.name AS restaurant_a, b.name AS restaurant_b, a.cuisine
FROM restaurant a
JOIN restaurant b ON a.cuisine = b.cuisine
AND a.restaurant_id < b.restaurant_id
ORDER BY a.cuisine;
```

Output:

restaurant_a	restaurant_b	cuisine
Dragon Wok	Wok Express	Chinese
Punjab Da Dhaba	Amritsari Kulcha House	North Indian
Udupi Sagar	Madras Filter Cafe	South Indian
(3 rows)		

20. Using a VIEW in a JOIN

Show the 5 biggest delivered orders by value, with customer name, using `v_order_total`.

Hint

A view is queried exactly like a table.

Solution

```
SELECT t.order_id, c.full_name AS customer, t.order_total
FROM v_order_total t
JOIN customer c ON c.customer_id = t.customer_id
```

```
WHERE t.status = 'Delivered'
ORDER BY t.order_total DESC
LIMIT 5;
```

Output:

order_id	customer	order_total
26	Aditya Rao	1850.00
20	Ananya Iyer	1246.00
2	Rohan Mehta	1120.00
18	Rohan Mehta	1047.00
21	Tanvi Kapoor	898.00

(5 rows)

Level 4 — Subqueries

Queries inside queries: scalar, `IN`, correlated, `EXISTS` / `NOT EXISTS`, and derived tables in `FROM`.

21. Scalar subquery

Which dish is the most expensive on the whole platform? Use a subquery, not `LIMIT`.

Hint

```
WHERE price = (SELECT MAX(price) FROM ...)
```

Solution

```
SELECT name, price
FROM menu_item
WHERE price = (SELECT MAX(price) FROM menu_item);
```

Output:

name	price
Pepperoni Pizza	499.00
(1 row)	

22. IN subquery

Which customers have ordered from 'Biryani Darbar'?

Hint

The inner query returns a list of customer IDs.

Solution

```
SELECT full_name
FROM customer
WHERE customer_id IN (SELECT fo.customer_id
                      FROM food_order fo
                      JOIN restaurant r ON r.restaurant_id = fo.restaurant_id
                      WHERE r.name = 'Biryani Darbar')
ORDER BY full_name;
```

Output:

full_name
Aarav Sharma
Aditya Rao
Karan Malhotra
Neha Gupta
Siddharth Joshi
Sneha Kulkarni
(6 rows)

23. Correlated subquery

List dishes priced **above their own restaurant's average** price.

Hint

The inner query refers to the outer row (`m2.restaurant_id = m.restaurant_id`), so it reruns for each row.

Solution

```
SELECT m.restaurant_id, m.name, m.price
FROM   menu_item m
WHERE  m.price > (SELECT AVG(m2.price)
                  FROM   menu_item m2
                  WHERE  m2.restaurant_id = m.restaurant_id)
ORDER BY m.restaurant_id, m.price DESC;
```

Output:

restaurant_id	name	price
1	Butter Chicken	340.00
1	Paneer Tikka	290.00
1	Dal Makhani	260.00
2	Rava Uttapam	130.00
2	Masala Dosa	120.00
3	Chicken Manchurian	260.00
3	Chilli Paneer	250.00
3	Schezwan Fried Rice	230.00
4	Pepperoni Pizza	499.00
4	Margherita Pizza	399.00
4	Penne Arrabbiata	349.00
5	Mutton Biryani	420.00
5	Chicken Dum Biryani	320.00
5	Chicken Seekh Kebab	280.00
6	Pav Bhaji	140.00
6	Sev Puri	70.00
7	Paneer Butter Masala	280.00

7	Pindi Chole	180.00
8	Ghee Roast Dosa	160.00
8	Ven Pongal	130.00
8	Curd Rice	120.00
9	Chicken Fried Rice	250.00
9	Chicken Hakka Noodles	240.00
10	Grilled Chicken Steak	450.00
10	Quinoa Buddha Bowl	380.00
10	Classic Caesar Salad	320.00
11	Prawn Malvani Curry	420.00
11	Surmai Fry	380.00

(28 rows)

24. NOT EXISTS

Which restaurants that *have* orders never had one cancelled?

Hint

EXISTS for 'has orders', plus NOT EXISTS for 'no cancelled order'.

Solution

```
SELECT r.name
FROM   restaurant r
WHERE  EXISTS      (SELECT 1 FROM food_order fo
                    WHERE fo.restaurant_id = r.restaurant_id)
      AND NOT EXISTS (SELECT 1 FROM food_order fo
                    WHERE fo.restaurant_id = r.restaurant_id
                      AND fo.status = 'Cancelled')
ORDER BY r.name;
```

Output:

```
      name
-----
Amritsari Kulcha House
Biryani Darbar
```

```
La Pizzeria Napoli
Madras Filter Cafe
Mumbai Chaat Corner
Punjab Da Dhaba
Udupi Sagar
Wok Express
(8 rows)
```

25. Derived table (subquery in FROM)

Which customers spent more than the average customer, counting delivered orders only?

Hint

Compute spend per customer in a subquery, then compare it with the average of that subquery.

Solution

```
SELECT c.full_name, s.spent
FROM   (SELECT customer_id, SUM(order_total) AS spent
        FROM   v_order_total
        WHERE  status = 'Delivered'
        GROUP BY customer_id) AS s
JOIN   customer c ON c.customer_id = s.customer_id
WHERE  s.spent > (SELECT AVG(spent)
                  FROM (SELECT SUM(order_total) AS spent
                        FROM   v_order_total
                        WHERE  status = 'Delivered'
                        GROUP BY customer_id) AS x)
ORDER BY s.spent DESC;
```

Output:

full_name	spent
Rohan Mehta	2797.00

Aarav Sharma	2720.00
Aditya Rao	2120.00
Karan Malhotra	1920.00
Tanvi Kapoor	1916.00
Meera Pillai	1850.00
Rahul Verma	1848.00

(7 rows)

Level 5 — Advanced (CTEs, window functions, CASE, dates, transactions)

WITH (CTEs), CASE, date functions, window functions (RANK, DENSE_RANK, running SUM, LAG), transactions, views and EXPLAIN.

26. CASE expressions

Label each delivered order as Small (< ₹400), Medium (₹400–₹799) or Large (₹800+), and count how many fall in each bucket.

Hint

CASE WHEN ... THEN ... END inside a CTE, then GROUP BY the label.

Solution

```
WITH sized AS (  
    SELECT order_id, order_total,  
           CASE WHEN order_total < 400 THEN 'Small'  
                WHEN order_total < 800 THEN 'Medium'  
                ELSE 'Large' END  
    FROM   v_order_total  
    WHERE  status = 'Delivered'  
)  
SELECT size, COUNT(*) AS orders  
FROM   sized
```

```
GROUP BY size
ORDER BY MIN(order_total);    -- Small, Medium, Large
```

Output:

size	orders
Small	12
Medium	25
Large	8

(3 rows)

27. Date functions

How many orders were placed on each day of the week?

Hint

`to_char(order_time, 'Dy')` gives the day name.
`EXTRACT (ISODOW ...)` gives a number you can sort by.

Solution

```
SELECT to_char(order_time, 'Dy') AS weekday, COUNT(*) AS
FROM   food_order
GROUP BY to_char(order_time, 'Dy'), EXTRACT (ISODOW FROM
ORDER BY EXTRACT (ISODOW FROM order_time);
```

Output:

weekday	orders
Mon	4
Tue	8
Wed	8
Thu	9
Fri	8

Sat		5
Sun		7

(7 rows)

28. Window function: DENSE_RANK

Rank customers by delivered spend. Customers with equal spend share a rank. Show the top 5.

Hint

```
DENSE_RANK() OVER (ORDER BY ... DESC)
```

Solution

```
SELECT c.full_name,
       SUM(t.order_total)
       DENSE_RANK() OVER (ORDER BY SUM(t.order_total) DESC)
FROM   v_order_total t
JOIN   customer c ON c.customer_id = t.customer_id
WHERE  t.status = 'Delivered'
GROUP BY c.customer_id, c.full_name
ORDER BY spend_rank
LIMIT 5;
```

Output:

full_name		spent		spend_rank
-----	+	-----	+	-----
Rohan Mehta		2797.00		1
Aarav Sharma		2720.00		2
Aditya Rao		2120.00		3
Karan Malhotra		1920.00		4
Tanvi Kapoor		1916.00		5

(5 rows)

29. Window function: running total

Show daily delivered revenue for 1–10 Sep 2026, with a running (cumulative) total.

Hint

`SUM(...) OVER (ORDER BY day)`. Aggregate per day first, inside a CTE.

Solution

```
WITH daily AS (  
    SELECT order_time::DATE AS day, SUM(order_total) AS r  
    FROM    v_order_total  
    WHERE   status = 'Delivered'  
           AND order_time < '2026-09-11'  
    GROUP BY order_time::DATE  
)  
SELECT day, revenue,  
       SUM(revenue) OVER (ORDER BY day) AS running_total  
FROM   daily  
ORDER BY day;
```

Output:

day	revenue	running_total
2026-09-01	1200.00	1200.00
2026-09-02	1620.00	2820.00
2026-09-03	1918.00	4738.00
2026-09-04	880.00	5618.00
2026-09-05	600.00	6218.00
2026-09-06	1070.00	7288.00
2026-09-07	1240.00	8528.00
2026-09-08	1307.00	9835.00
2026-09-09	210.00	10045.00
2026-09-10	810.00	10855.00
(10 rows)		

30. Window function: share of a group

For Biryani Darbar's delivered orders, show each order's value and its percentage of the restaurant's total.

Hint

`SUM(order_total) OVER (PARTITION BY restaurant_id)` gives the group total on every row.

Solution

```
SELECT order_id, order_total,
       ROUND(100 * order_total
             / SUM(order_total) OVER (PARTITION BY restaurant_id
                                     ORDER BY order_total DESC)) AS pct_of_restaurant
FROM   v_order_total
WHERE  status = 'Delivered' AND restaurant_id = 5
ORDER  BY order_total DESC;
```

Output:

order_id	order_total	pct_of_restaurant
26	1850.00	35.1
22	860.00	16.3
24	740.00	14.0
27	740.00	14.0
23	600.00	11.4
25	480.00	9.1
(6 rows)		

31. Window function: LAG

For customer #1 (Aarav), show each order, the previous order's time, and the days between them.

Hint

```
LAG(order_time) OVER (PARTITION BY customer_id ORDER BY order_time)
```

Solution

```
SELECT order_id,
       order_time::DATE
       LAG(order_time::DATE) OVER (ORDER BY order_time)
       order_time::DATE - LAG(order_time::DATE) OVER (ORD
FROM   food_order
WHERE  customer_id = 1
ORDER  BY order_time;
```

Output:

order_id	ordered_on	previous_order	days_gap
1	2026-09-02		
33	2026-09-04	2026-09-02	2
24	2026-09-11	2026-09-04	7
6	2026-09-22	2026-09-11	11

(4 rows)

32. Transactions & why unit_price is stored

Raise every Punjab Da Dhaba menu price by 10% inside a transaction. Show that past order totals **do not change**, then undo the change with ROLLBACK.

Hint

```
BEGIN; UPDATE ...; SELECT ...; ROLLBACK; Past orders keep the unit_price they were charged.
```

Solution

```
SELECT order_id, order_total FROM v_order_total
WHERE  restaurant_id = 1 ORDER BY order_id;  -- BEFORE
```

```

BEGIN;
UPDATE menu_item SET price = ROUND(price * 1.10, 2) WHERE

SELECT name, price AS new_menu_price FROM menu_item
WHERE  restaurant_id = 1 ORDER BY menu_item_id;

SELECT order_id, order_total FROM v_order_total
WHERE  restaurant_id = 1 ORDER BY order_id;    -- AFTER: i

ROLLBACK;    -- nothing is saved

```

Output:

order_id	order_total
-----+-----	
1	760.00
2	1120.00
3	630.00
4	380.00
5	810.00
6	630.00
48	460.00
(7 rows)	
name	new_menu_price
-----+-----	
Butter Chicken	374.00
Dal Makhani	286.00
Paneer Tikka	319.00
Butter Naan	66.00
Sweet Lassi	99.00
(5 rows)	
order_id	order_total
-----+-----	
1	760.00
2	1120.00
3	630.00

4		380.00
5		810.00
6		630.00
48		460.00

(7 rows)

33. Create and use a VIEW

Create a view `v_veg_menu` with each veg dish and its restaurant name, then query the 5 cheapest. Drop the view afterwards.

Hint

```
CREATE VIEW ... AS SELECT ...
```

Solution

```
CREATE VIEW v_veg_menu AS
SELECT r.name AS restaurant, m.name AS dish, m.price
FROM   menu_item m
JOIN   restaurant r ON r.restaurant_id = m.restaurant_id
WHERE  m.is_veg;

SELECT * FROM v_veg_menu ORDER BY price, dish LIMIT 5;

DROP VIEW v_veg_menu;
```

Output:

restaurant	dish	price
Mumbai Chaat Corner	Vada Pav	30.00
Mumbai Chaat Corner	Masala Chaas	40.00
Udupi Sagar	Filter Coffee	50.00
Punjab Da Dhaba	Butter Naan	60.00
Madras Filter Cafe	Madras Filter Coffee	60.00

(5 rows)

34. Reading a query plan

Use EXPLAIN to check whether the cuisine-and-location search can use `idx_restaurant_cuisine_location`.

Hint

With only 11 rows, PostgreSQL may choose a Seq Scan anyway, because reading one page is cheaper than using an index. `SET enable_seqscan = off` forces it to show the index path. See `05_index_demo.sql` for the 200,000-row version.

Solution

```
SET enable_seqscan = off;    -- for this session only: show
EXPLAIN (COSTS OFF)
SELECT name FROM restaurant
WHERE  cuisine = 'Chinese' AND location = 'Vashi';
RESET enable_seqscan;
```

Output:

```
                                QUERY PLAN
-----
Index Scan using idx_restaurant_cuisine_location on restaurant
    Index Cond: (((cuisine)::text = 'Chinese'::text) AND (location = 'Vashi'))
(2 rows)
```

Self-check challenges (no solutions)

These have no answers given. Check each one by reasoning about the data, or by comparing it with the outputs in `outputs/04_queries_output.txt` where they overlap.

1. List each cuisine with its **number of veg dishes** and **number of non-veg dishes** in two columns. (*Hint: `COUNT(*) FILTER`*)

(WHERE is_veg), or SUM(CASE ...).)

2. Find the delivery agent with the **fastest average delivery time**, counting only agents with at least 5 deliveries.
3. For every restaurant, show its **best day by revenue**. (*Hint: aggregate per restaurant per day, then RANK() OVER (PARTITION BY restaurant_id ...).*)
4. Find customers who ordered from **at least 3 different restaurants**.
5. Write an INSERT for a new order with 2 dishes and a payment, inside one transaction. Then try adding a dish from a *different* restaurant and read the error the trigger raises.
6. Rewrite exercise 24 (NOT EXISTS) using LEFT JOIN ... IS NULL. Do you get the same 8 restaurants?
7. Using EXPLAIN ANALYZE, compare exercise 23 (correlated subquery) with a version that uses AVG(price) OVER (PARTITION BY restaurant_id).
8. Which **hour of the day** gets the most orders? (*Hint: EXTRACT(HOUR FROM order_time).*)